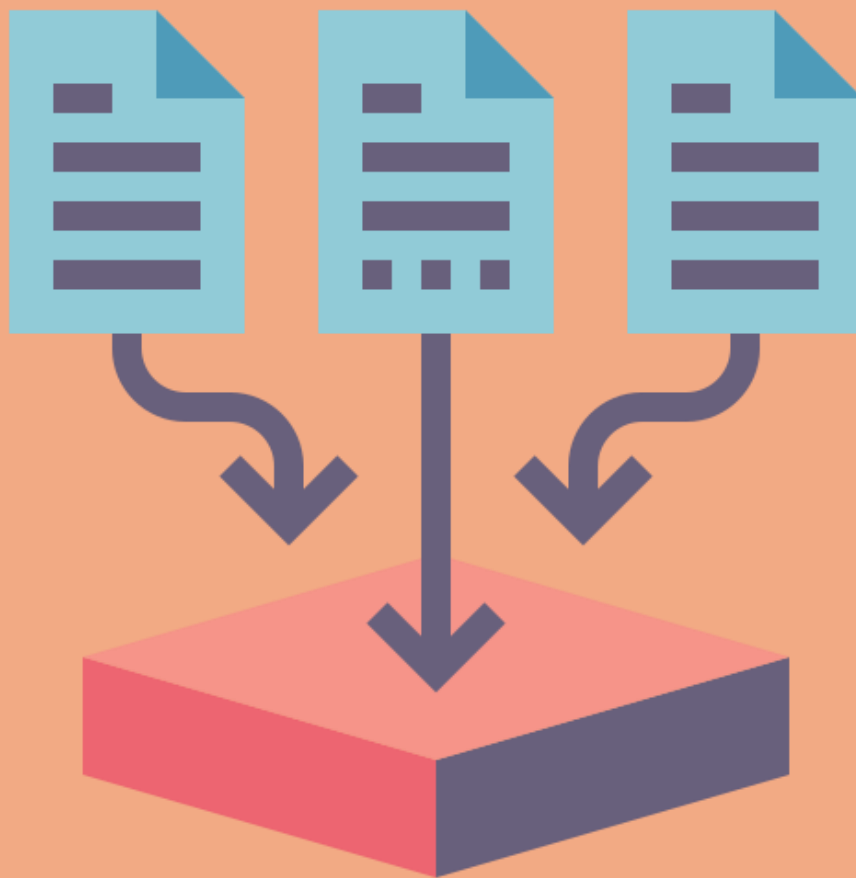


Serialization & Deserialization

Python Guide



Contact:



Email: vladplan8@gmail.com



LinkedIn: www.linkedin.com/in/vladyslav-urdenko

Hello everyone! This is the next episode in our Python guide series. In today's lesson, we're diving into a very important topic: **Serialization and Deserialization**.

The material will be a bit more advanced than in the previous lessons, so I highly recommend checking out the earlier ones if you haven't already. I've structured this program to include lots of real practice and hands-on examples - so make sure you have VS Code open and ready to go!

Overall, just enjoy the process and level up your brain 🧠

Let's get started!



Today's Agenda:

- What is it and why do we need it?

We'll break down the core concept in simple terms and explain why it's essential in real-world applications.

- Where is it used?

From backend systems and APIs to data storage and application state management - we'll explore practical use cases.

- `strip()` and `split()` - our reliable helpers

Learn how to clean and process data effectively using these essential string methods.

- A simple real-world example

We'll start with a basic, hands-on example to clearly understand how everything works.

- Improving our Cars Management system

We'll enhance an existing project by applying newly learned concepts.

- Adding OOP to our program

Introducing classes and objects to make our code cleaner, more structured, and scalable.

- A complete application using OOP and everything we've learned

We'll put it all together and build a realistic application from scratch.

- What is JSON and why is it so important?

Understanding the data format that powers modern applications and communication.

- Solving problems using serialization, deserialization, and JSON

The Necessity of Files

At its core, computer memory (RAM) is volatile. When you run a program, define variables, or build complex objects, they only exist as long as the program is running. Once the program closes or the computer shuts down, that data vanishes.

Files serve as the bridge between sessions. They provide persistence, allowing data to survive beyond the lifespan of the process that created it. Whether it is a simple text document, a saved game state, or a complex configuration, files allow us to "freeze" the state of our software and "thaw" it out later.



The Core Concept: Serialization & Deserialization

To save data to a file, we cannot simply shove a Python object (like a complex Dictionary or a custom Class) directly onto the hard drive. The hard drive only understands streams of bytes.

Serialization (The "Packing" Phase):

- This is the process of converting complex data structures (objects, lists, dictionaries) into a format that can be stored (in a file) or transmitted (across a network).

Think of it as: Taking a piece of IKEA furniture (your code object) and dismantling it into a flat-pack box (a byte stream or text string) so it can be shipped.

Deserialization (The "Unpacking" Phase):

- This is the reverse process. It involves reading the stored data from the file and reconstructing the original object in memory so the code can be used again.

Think of it as: Opening the flat-pack box and assembling the furniture back into its usable 3D form.

The Role of `strip()` and `split()` in Text Serialization

When we serialize data into text formats (like CSV, logs, or custom text files), Python reads raw strings. This is where `strip()` and `split()` become critical tools for "cleaning" and "parsing" the data during Deserialization.

Split

Split - breaks a string into a list of smaller strings (substrings) based on a specified separator.

part1 | part2 | part3
apple,banana,orange



a list

In code:

```
text = "apple,banana,cherry"
fruits = text.split(",")
print(fruits)
# Output: ['apple', 'banana', 'cherry']
```

And now it is different part, you see in future why this is useful. Move to strip.

Strip here:

Strip - removes characters from the beginning (left) and end (right) of a string.

TEXTTEXTTEXT $\longrightarrow$ TEXTTEXT

In code:

```
text = "  Hello World!  "
clean_text = text.strip()
print(clean_text)
# Output: "Hello World!" (spaces removed)
```

Real-World Use Cases

Programmers use serialization constantly. Here are three common scenarios:

API Communication (JSON):

When your mobile app talks to a server (like Instagram fetching your feed), the server serializes database objects into JSON text strings. Your phone receives them, deserializes them, and shows you the pictures.

Game Save States:

When you save a game, the code serializes your character's location, health, and inventory into a binary file. When you load the game, it deserializes that file to put you back exactly where you were.

Machine Learning Models:

Data scientists spend days training a model. They serialize the mathematical weights of the model to a file (often using pickle in Python) so they don't have to retrain it every time they want to make a prediction.

The Future: Scaling to Databases

Files are excellent for simple persistence, but they have limits. You generally load a whole file into memory to read it. As your data grows to millions of records, files become slow and hard to search.

The Transition to Databases (DB):

- Instead of `split()`-ing lines in a text file, a Database (like PostgreSQL or SQLite) handles the structure for you. You query specific rows without loading the entire dataset.
- Sometimes, programmers store serialized data inside a database. They might take a complex Python object, serialize it into bytes, and store it in a single "BLOB" (Binary Large Object) column in the database. This allows them to save complex logic that doesn't fit neatly into standard table rows.

And after all, let's practice!!



Base Car Management

Task 1: The Concept of Serialization

Goal: Convert data from computer memory (variables) into a single string format.

Scenario: We have information about a car in separate variables. We need to glue them together so they can be sent or stored as one unit.

Note:

In this task, we concatenate variables into one string. We use the pipe character `|` as a separator. We avoid using commas, because sometimes data itself contains commas (e.g., a car model description might be "Standard, with AC"), which would confuse the program later.

```

car_management > main.py > ...
1  # Task 1. Basic serialization
2
3  car_make = "BMW"
4  car_model = "X5"
5  car_year = 2020
6  car_price = 50000.0
7
8  print(f"Original data: {car_make}, {car_model}, {car_year}, {car_price}")
9
10 # serialize: convert data to specific string format
11 # we choose the separator as '|'
12 serialized_data = f"{car_make}|{car_model}|{car_year}|{car_price}"
13
14 print("====Serialized output====")
15 print(f"Raw string: {serialized_data}")
16 print(f"Type: {type(serialized_data)}")

```

And output should be:

```

Original data: BMW, X5, 2020, 50000.0
====Serialized output====
Raw string: BMW|X5|2020|50000.0
Type: <class 'str'>
PS V:\PythonLAB_Lesson>

```

Task 2: The Concept of Deserialization

Goal: Take a raw string and convert it back into usable Python variables.

Scenario: We received a string representing a car. We need to extract the price to calculate a discount, but right now, the price is just text.

Here we use the **split('|')** function. It acts like a pair of scissors. It looks for every | in the string and cuts the string at those points.

Result: It returns a list of smaller strings: "BMW|X5" becomes ["BMW", "X5"].

This is crucial because computers read the raw string as one giant block of text. split helps us separate the "Make" from the "Model."

```

car_management > main.py > ...
1  # Task 2. Basic deserialization (String -> Data)
2
3  # Incoming raw data
4  raw_data = "Mercedes-Benz|C-Class|2020|35000"
5
6  # Deserialize: Split the raw back to variables
7  parts = raw_data.split("|")
8
9  # Assign variables and fix data types
10 make = parts[0]
11 model = parts[1]
12 year = int(parts[2]) # Convert string to integer
13 price = float(parts[3]) # Convert string to float
14
15 # Restored data
16
17 print(f"Restored data: \nMake: {make}, \nModel: {model}, \nYear: {year}, \nPrice: {price}")
18

```

Output:

```

Restored data:
Make: Mercedes-Benz,
Model: C-Class,
Year: 2020,
Price: 35000.0

```

Task 3: Persistence (Writing to File)

Goal: Save multiple cars into a physical file on the hard drive.

Scenario: We have a list of cars. We need to save them, so we don't lose data when the program closes.

Also check that you are in the same folder. File main and garage.txt should be in the same directory. Because in other cases it would be difficult to write correctly.

Note:

When writing to a file, we must manually add the `\n` (newline character) at the end of every line. If we forget this, all our cars will be written on one single long line, making it impossible to read them back to individually later.


```

car_management > main.py > ...
1  # Task 2. Writing to a file (Output)
2
3  # A list of cars. Dictionary
4  garage = [
5      {"make": "Toyota", "model": "Camry", "year": 2019, "price": 24000},
6      {"make": "BMW", "model": "X5", "year": 2021, "price": 60000},
7      {"make": "Audi", "model": "A4", "year": 2018, "price": 30000}
8  ]
9
10 filename = "garage.txt"
11
12 # Open file in write mode ('w')
13 with open(filename, 'w', encoding="utf-8") as file:
14     print(f"Writing to {filename}...")
15     for car in garage:
16         # Create a line of text for each car Make|Model|Year|Price
17         line = f"{car['make']}|{car['model']}|{car['year']}|{car['price']}\n"
18         file.write(line + "\n")
19         print(f"Written: {line}")
20
21 print(f"Done. Check the file folder!!")
22

```

Garage.txt

```

car_management > garage.txt
1  Toyota|Camry|2019|24000
2
3  BMW|X5|2021|60000
4
5  Audi|A4|2018|30000
6
7

```

Output:

```

Writing to garage.txt...
Written: Toyota|Camry|2019|24000

Written: BMW|X5|2021|60000

Written: Audi|A4|2018|30000

Done. Check the file folder!!

```

Task 4: Restoration (Reading from File)

Goal: Read the file line-by-line and reconstruct the list of cars.

Scenario: The program just started. We need to load the database we saved on Task 3.

Note: The Role of `strip()`

When Python reads a line from a file, it includes the invisible "Enter" key press at the end (the `\n`).

If we read the price, we might get `"35000\n"` instead of `"35000"`. This can cause errors when converting to numbers, but the Solution (`strip()`) - this function removes whitespace and invisible characters from both ends of the string. It cleans the data before we process it.

```
car_management > main.py > ...
1 # Task 4. Reading from a file (Input)
2 filename = "garage.txt"
3 restored_garage = []
4
5 print(f"Reading from {filename}...")
6 # Open file in read mode ('r')
7 with open(filename, 'r', encoding="utf-8") as file:
8     for line in file:
9         # First step: clean file from \n
10         clean_line = line.strip()
11         # If the line is empty, we skip it and move to the next one
12         if not clean_line:
13             continue
14         # Second step: split line into parts by '|'
15         parts = clean_line.split('|')
16         # Third step: create a dictionary for each car
17         car = {
18             "make" : parts[0],
19             "model" : parts[1],
20             "year" : int(parts[2]),
21             "price" : int(parts[3])
22         }
23         # append use to add the car to the restored_garage list
24         restored_garage.append(car)
25
26 # Let's output the restored garage to check if everything is correct
27 print("DB has been loaded...")
28 for car in restored_garage:
29     print(f"Car : {car['make']} {car['model']} ({car['year']}) - {car['price']}€")
30
31
```

Output:

```
Reading from garage.txt...
DB has been loaded...
Car : Toyota Camry (2019) - 24000€
Car : BMW X5 (2021) - 60000€
Car : Audi A4 (2018) - 30000€
```

Task 5: The "Pro" Level (OOP Encapsulation)

Goal: Refactor the code using Object-Oriented Programming. The class itself should handle its own serialization.

Scenario: In a real project, we don't want split and formatting code scattered everywhere. We hide it inside methods.

Note:

This approach uses Encapsulation. By putting the split and formatting logic inside the Class (using methods like `to_file_format`), the rest of our program doesn't need to know how the data is saved, just that it is saved. This makes the code cleaner and easier to update.

```
car_management > main.py > ...
1  # Task 5. OOP Approach
2
3  class Car:
4      def __init__(self, make, model, year, price):
5          self.make = make
6          self.model = model
7          self.year = year
8          self.price = price
9
10     def __str__(self):
11         return f"{self.make} {self.model} {self.year}"
12
13     # Method to serialize (Object -> String)
14     def to_file_format(self):
15         return f"{self.make}|{self.model}|{self.year}|{self.price}"
16     # Static method to deserialize (String -> Object)
17     @staticmethod
18     def from_file_format(line):
19         parts = line.strip().split("|")
20         # Return a new Car object
21         return Car(parts[0], parts[1], int(parts[2]), float(parts[3]))
22
23     # Usage example
24     # Creating and saving
25     myCar = Car("Toyota", "Corolla", 2020, 20000)
26     serialized = myCar.to_file_format()
27     print(f"Serialized for file: '{serialized}'")
28
29     # Imagine we write this to a file and read it back
30
31     # Loading and deserializing
32     loaded_string = "Porsche|911|2021|90000"
33     restored_car = Car.from_file_format(loaded_string)
34
35     print(f"Restored Car: {restored_car}")
36     print(f"Accessing Property: It costs {restored_car.price}€")
37
```

Output:

```
Serialized for file: 'Toyota|Corolla|2020|20000'
Restored Car: Porsche 911 2021
Accessing Property: It costs 90000.0€
```



Object-Oriented Programming (OOP): The Car Model

Thinking in Objects

Up until now, we have been writing code linearly (Procedural Programming). This works for small scripts, but it gets messy for big projects.

- OOP allows us to organize code by modeling real-world things.
- Class: The Blueprint or Template (The technical drawing of a car).

Object: The actual thing created from the blueprint (The actual metallic BMW sitting in your driveway).

Master Task: The Complete Car Management System (OOP)

The Goal

Create a self-contained Car Class that handles everything about a vehicle: its data (attributes), its behavior (methods), and its ability to save/load itself (serialization).

Our Plan:

Your dealership needs a robust software module. Instead of writing loose functions and variables scattered across a file, you must build a "Black Box" (the Class).

The main program should not need to know how to format the text file or how a car drives. It should simply tell the Car object: "Save yourself" or "Create a new car from this text line."

The Challenge Requirements 😊

You need to write a single class Car that satisfies all the following requirements:

- The Blueprint (`__init__`):
 - The class must accept make, model, year, and price when created.
- Constraint: Ensure year is an integer and price is a float.
- Human Readable Output (`__str__`):
 - When we print(`my_car`), it should display a nice string like "Toyota Corolla (2020)", not a memory address.
- Behavior (drive):
 - Add a method `drive()` that prints "Vroom! The [Make] [Model] is driving."
- Encapsulated Serialization (`to_file_format`):
 - The object must know how to convert itself into a string.
- Format: Pipe-separated values (Make|Model|Year|Price).

- Why? The main program shouldn't mess with string formatting manually.
- Encapsulated Deserialization (from_file_format):
 - Add a Static Method that takes a raw text line from a file and returns a ready-to-use Car object.
- Logic: It must strip() the whitespace, split() by the pipe |, and convert numbers back to int/float.

Here the solution:

```
car_management > main.py > ...
1  # Task 6. The full cycle (List of objects)
2
3  import os
4
5  class Car:
6      def __init__(self, make, mode, year, price):
7          self.make = make
8          self.mode = mode
9          self.year = int(year)
10         self.price = float(price)
11
12
13         def drive (self):
14             return f"Vroom! The {self.make} {self.mode} is driving."
15         # serialization logic, incapsulation inside the class
16         def to_file_format(self):
17             return f"{self.make}|{self.mode}|{self.year}|{self.price}"
18         # Static method: Hey class, take this string and give me a Car object
19         @staticmethod
20         def from_file_format(data):
21             parts = data.strip().split("|")
22             # Validation, good practice
23             if len(parts) != 4:
24                 raise ValueError("Invalid data format")
25             return Car(parts[0], parts[1], parts[2], parts[3])
26
27
28     filename = "cars.txt"
29
```

Second part:

```
27
28     filename = "cars.txt"
29
30     # Saving data
31     print("===Creating and saving data===")
32     # List of objects
33     inventory = [
34         Car("Toyota", "Camry", 2020, 24000),
35         Car("Honda", "Civic", 2019, 22000),
36         Car("Ford", "Mustang", 2021, 35000)
37     ]
38     # Writing to file
39     with open(filename, "w", encoding="utf-8") as file:
40         for car in inventory:
41             #We don't manually format strings here. We ask objects to do it for us. Encapsulation.
42             line = car.to_file_format()
43             file.write(line + "\n")
44             print(f"Saved: {line}")
45
46     print("\n(Simulating program restart...)\n")
47     inventory = [] # Clearing the list to simulate program restart
48
```

Third part:

```

48
49 # Loading data
50 print("===Loading data from file===")
51 restored_inventory = []
52
53 if os.path.exists(filename):
54     with open(filename, 'r', encoding="utf-8") as file:
55         for line in file:
56             try:
57                 new_car_object = Car.from_file_format(line)
58                 restored_inventory.append(new_car_object)
59             except ValueError as e:
60                 print(f"Error processing line: {line.strip()} - {e}")
61 # Verification
62 print("===Restored Inventory===")
63 for car in restored_inventory:
64     # we can call methods like drive()
65     print(f"{car.make} says: {car.drive()}")
66
67 # calculate tax (using object properties)
68 for car in restored_inventory:
69     tax = car.price * 0.1
70     print(f"Tax for {car.make} {car.model}: ${tax:.2f}")
71

```

And output:

```

===Creating and saving data===
Saved: Toyota|Camry|2020|24000.0
Saved: Honda|Civic|2019|22000.0
Saved: Ford|Mustang|2021|35000.0

(Simulating program restart...)

===Loading data from file===
===Restored Inventory===
Toyota says: Vroom! The Toyota Camry is driving.
Honda says: Vroom! The Honda Civic is driving.
Ford says: Vroom! The Ford Mustang is driving.
Tax for Toyota Camry: $2400.00
Tax for Honda Civic: $2200.00
Tax for Ford Mustang: $3500.00

```

Work with JSON: The Standard of the Web

Why JSON?

In our previous tasks, we invented our own format using pipes (|). This works, but it has a problem: Only our program understands it.

If you want to send data to a website, a mobile app, or another program, you need Universal Language.

JSON (JavaScript Object Notation) is that language. It looks almost exactly like a Python Dictionary, making it incredibly easy for us to use.

Task 1: Simple Serialization (The Hero Inventory)

Goal: Convert a Python dictionary into a JSON string.

Scenario: You are building an RPG game. You need to save the player's inventory (name, level, weapons) so they can resume the game later.

Note:

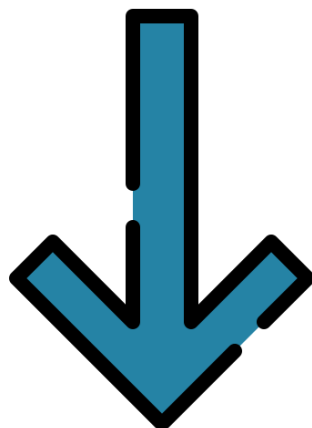
json.dumps() means "Dump to String". It takes a Python dictionary and turns it into a text string that looks like JavaScript.

Notice the difference, Python uses True (capital T). The JSON output uses true (lowercase t). This shows that serialization is a conversion process, not just copying.

Structure of the project:



First part of the solution here:



```
game_save.py X
serializing_1 > game_save.py > ...
1 import json
2
3 #Initial data (Dictionary)
4
5 hero_inventory = {
6     "name": "Aragorn",
7     "level": 20,
8     "is_alive": True,
9     "weapons": ["Anduril", "Dagger"],
10    "potions": {
11        "health": 5,
12        "mana": 2
13    }
14 }
15
16 # I guess you understand what I've wrote previously
17
18 # Initial object
19 print(f"Type: {type(hero_inventory)}")
20 print(hero_inventory)
21 print("\n")
22
23 # Serializing (Converting to JSON string)
24 # dumps() - dump string
25 # indent(4) - just for beautiful output
26 json_string = json.dumps(hero_inventory, indent=4)
27 print(f"Type: {type(json_string)}")
28 print(json_string)
29
30 # Now, we can save this JSON string to a file
31 # Check your console
32
```

Second part:

```
32
33 # And now, let's deserialize it back to a Python object
34 # loads() - load string
35 restored_inventory = json.loads(json_string)
36 print(f"Type: {type(restored_inventory)}")
37 print(f"Name: {restored_inventory['name']}")
38 print(f"Level: {restored_inventory['level']}")
39 print(f"Is Alive: {restored_inventory['is_alive']}")
40
41 # Check if data is the same
42 if hero_inventory == restored_inventory:
43     print("Success!! Data is identical young hero!!")
```

Master Task: The Persistent To-Do List (JSON)

The Goal

Build a professional-grade Task Manager Application that saves its data to a JSON file.

Unlike the simple text files we used before, this app must be able to handle complex objects and share data in a format compatible with web browsers and mobile apps.

The Scenario

You are a backend developer. Your front-end team needs a way to store user tasks. They have asked you to build a system where:

- Tasks are Python Objects (so we can add logic later).

- Data is stored as JSON (so the frontend can understand it).
- The code is modular (User Interface is separate from Data Logic).

The Challenge Requirements 😊

You need to create a project with two files: `task_manager.py` (logic) and `main.py` (interface).

Part A: The Logic (`task_manager.py`)

The Class (Task):

Create a class with:

- `title` (String)
- `is_done` (Boolean, default False)

The Adapter Pattern (`to_dict`):

The built-in `json` module doesn't know how to save your custom Task class.

- Requirement: Add a method `to_dict(self)` that returns a simple Python dictionary: `'title': '...', 'is_done': ...`.

The Reverse Adapter (`from_dict`):

- Requirement: Add a static method `from_dict(data)` that takes a dictionary and returns a new Task object.

File Persistence (save & load):

- Write two functions outside the class:
 - `save_tasks(tasks)`: Opens `tasks.json` and uses `json.dump()` to write the list of dictionaries.
 - `load_tasks()`: Opens `tasks.json`, uses `json.load()` to get the data, and converts it back into a list of Task objects using your `from_dict` method.

Bonus: Use a **try/except** block to handle the case where the file doesn't exist yet.

Part B: The Interface (main.py)

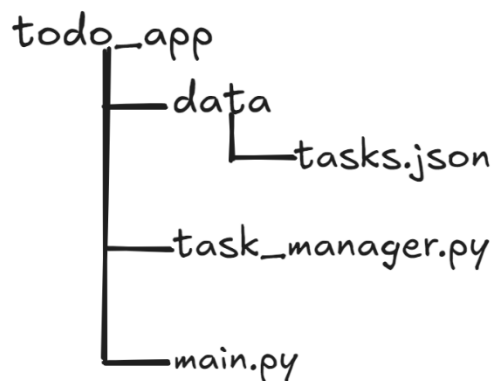
The User Loop:

Create a menu that runs forever (while True):

1. Add Task: Ask for a title, create a Task object, add it to the list, and Save.
2. Show Tasks: Load the list and print them (e.g., [x] Buy Milk).
3. Exit: Stop the program.

Solution:

Structure of the program:



Part 1, Task manager



```

todo_app > task_manager.py > ...
1  import json
2  import os
3
4  FILE_PATH = "data/tasks.json"
5  class Task:
6      def __init__(self, title, is_done=False):
7          self.title = title
8          self.is_done = is_done
9
10 # Method to convert class to dictionary for JSON serialization
11 def to_dict(self):
12     return {
13         "title": self.title,
14         "is_done": self.is_done
15     }
16 # Method to create a Task object from a dictionary
17 @staticmethod
18 def from_dict(data):
19     return Task(title=data["title"], is_done=data["is_done"])
20
21 def __repr__(self):
22     status = "[x]"
23     if self.is_done:
24         status = "[x]"
25     else:
26         status = "[ ]"
27     return f"{status} {self.title}"
28

```

Task manager, part 2:

```

28
29 def save_tasks(task_list):
30     # Serialize the list of Task objects
31     serialized_data = [task.to_dict() for task in task_list]
32     # Let's create a directory for our tasks if it doesn't exist
33     os.makedirs("data", exist_ok=True)
34     # Write our file
35     with open(FILE_PATH, "w", encoding="utf-8") as file:
36         json.dump(serialized_data, file, indent=4, ensure_ascii=False)
37     print("Saving data...")
38
39 def load_tasks():
40     # Load file, deserialize JSON string to list of dictionaries, and convert them to Task objects
41     if not os.path.exists(FILE_PATH):
42         return [] #means that if we don't have a file, empty list
43     try:
44         with open(FILE_PATH, "r", encoding="utf-8") as file:
45             data = json.load(file)
46             return [Task.from_dict(item) for item in data]
47     except json.JSONDecodeError:
48         print("Error: Can't read the file!!")
49     return []

```

Main logic:

Code below



```

todo_app > main.py > ...
1  from task_manager import Task, save_tasks, load_tasks
2
3  def main():
4      print("++++Welocome TO-DO++++")
5      # After loading, initially deserialize data
6      tasks = load_tasks()
7      print(f"Your tasks: {len(tasks)}")
8      while True:
9          print("\n1. Show tasks")
10         print("2. Add task")
11         print("3. Mark task as done")
12         print("4. Exit")
13
14         choice = input("Choose an option: ")
15         if choice == "1":
16             for idx, task in enumerate(tasks):
17                 print(f"{idx + 1}. {task}")
18
19         elif choice == "2":
20             title = input("Enter task title: ")
21             new_task = Task(title=title)
22             tasks.append(new_task)
23         elif choice == "3":
24             idx = int(input("Number of the task: ")) - 1
25             if 0 <= idx < len(tasks):
26                 tasks[idx].is_done = True
27             else:
28                 print("Invalid task number!!")
29         elif choice == "4":
30             # Serialize before exiting
31             save_tasks(tasks)
32             print("Goodbye!!")
33             break
34
35 if __name__ == "__main__":
36     main()

```

Output:

```

1. [x] Cook something
2. [ ] Study finnish

1. Show tasks
2. Add task
3. Mark task as done
4. Exit
Choose an option: 

```

Our data file

```

game_save.py  tasks.json  task_manager.py  main.py
todo_app > data > tasks.json > ...
1  [
2      {
3          "title": "Cook something",
4          "is_done": true
5      },
6      {
7          "title": "Study finnish",
8          "is_done": false
9      }
10 ]

```